

Algèbre 1 : Chapitre I - Calculs algébriques

Aymen Ben Brik

Chapitre 1

Calculs algébriques

1.1 Sommes et produits finis

1.1.1 Motivation

Dans de nombreux domaines tels que l'analyse de données, l'algorithmique ou les probabilités, nous sommes confrontés à la nécessité d'agréger ou de multiplier de grandes quantités de valeurs. L'écriture explicite de ces opérations (ex : $x_1 + x_2 + \dots + x_{1000}$) devient vite illisible et impraticable. L'introduction des symboles de sommation ($\sum$) et de produit ($\prod$) permet non seulement de compacter l'écriture, mais surtout de manipuler ces blocs de données de manière formelle et rigoureuse pour en extraire de nouvelles propriétés.

1.1.2 Approche par problème : L'astuce du jeune Gauss

Problème : Comment calculer rapidement la somme des 100 premiers nombres entiers : $S = 1 + 2 + 3 + \dots + 99 + 100$?

Résolution : Une approche naïve consisterait à additionner terme à terme. Cependant, si l'on écrit cette somme à l'endroit, puis à l'envers, et qu'on les additionne verticalement :

$$\begin{array}{r} S = 1 + 2 + \dots + 99 + 100 \\ S = 100 + 99 + \dots + 2 + 1 \\ \hline 2S = 101 + 101 + \dots + 101 + 101 \end{array}$$

Puisqu'il y a 100 termes, on obtient $2S = 100 \times 101$, d'où $S = 5050$. Ce problème met en évidence le besoin d'un formalisme (les changements d'indices) pour généraliser ce type de raisonnement à tout entier n .

1.1.3 Définitions

Soient $n, p \in \mathbb{N}$ tels que $p \leq n$, et $(a_k)_{p \leq k \leq n}$ une suite de nombres réels ou complexes.

Définition 1.1 (Somme) : On note par la lettre grecque majuscule Sigma ($\sum$) la somme des éléments a_k pour l'indice k allant de p à n :

$$\sum_{k=p}^n a_k = a_p + a_{p+1} + \dots + a_n$$

Remarque : La variable k est une variable "muette". Elle n'existe qu'à l'intérieur de la somme. $\sum_{k=1}^n a_k = \sum_{i=1}^n a_i$.

Définition 1.2 (Produit) : On note par la lettre grecque majuscule Pi ($\prod$) le produit de ces mêmes éléments :

$$\prod_{k=p}^n a_k = a_p \times a_{p+1} \times \cdots \times a_n$$

Convention : Une somme vide (si $p > n$) vaut 0. Un produit vide vaut 1.

1.1.4 Exemples résolus (Difficulté ascendante)

Exemple 1 (Développement simple - Facile) : Développer $\sum_{k=2}^5 k^2$. *Correction :* On remplace successivement k par 2, 3, 4 et 5.

$$\sum_{k=2}^5 k^2 = 2^2 + 3^2 + 4^2 + 5^2 = 4 + 9 + 16 + 25 = 54$$

Exemple 2 (Changement d'indice - Intermédiaire) : Exprimer $S = \sum_{k=1}^n a_{k+2}$ sous la forme d'une somme dont le terme général est a_j . *Correction :* Posons $j = k + 2$. Quand $k = 1$, $j = 3$. Quand $k = n$, $j = n + 2$. Donc, $S = \sum_{j=3}^{n+2} a_j$.

Exemple 3 (Somme télescopique - Difficile) : Calculer $S_n = \sum_{k=1}^n \left(\frac{1}{k} - \frac{1}{k+1}\right)$. *Correction :* En développant les premiers et derniers termes, on observe des annulations en cascade :

$$S_n = \left(1 - \frac{1}{2}\right) + \left(\frac{1}{2} - \frac{1}{3}\right) + \cdots + \left(\frac{1}{n} - \frac{1}{n+1}\right)$$

Tous les termes intermédiaires s'annulent. Il ne reste que le premier et le dernier :

$$S_n = 1 - \frac{1}{n+1} = \frac{n}{n+1}$$

1.1.5 Exercices pratiques avec Python

Les sommes mathématiques se traduisent naturellement par des boucles `for` en développement logiciel. Python propose également des fonctions intégrées très performantes pour l'analyse de tableaux de données.

```

1  # Exercice : Calculer la somme des carres des 10 premiers entiers
2
3  # Methode 1 : Approche algorithmique classique (boucle for)
4  somme_boucle = 0
5  for k in range(1, 11): # k va de 1 a 10
6      somme_boucle += k**2
7  print(f"Resultat avec boucle : {somme_boucle}")
8
9  # Methode 2 : Approche "Pythonique" (comprehension de liste)
10 # Tres utile pour manipuler rapidement des donnees
11 somme_rapide = sum([k**2 for k in range(1, 11)])
12 print(f"Resultat avec sum() : {somme_rapide}")

```

1.1.6 Exercices à faire

1. **Exercice 1 :** Calculer la valeur exacte de $\prod_{k=1}^4 (2k)$.
2. **Exercice 2 :** En utilisant la relation de Chasles, simplifier l'expression : $\sum_{k=1}^{100} k^3 - \sum_{k=1}^{98} k^3$.
3. **Exercice 3 :** Montrer par le calcul que $\sum_{k=0}^n (2k+1) = (n+1)^2$.
4. **Exercice 4 (Code) :** Écrire une fonction Python `factorielle(n)` qui calcule $n! = \prod_{k=1}^n k$ à l'aide d'une boucle.